

```
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 // Structure for tree node
6 struct Node
7 {
8     int data;
9     struct Node *left;
10    struct Node *right;
11 };
12
13 // Create new node
14 struct Node* createNode(int value)
15 {
16     struct Node *newNode;
17
18     newNode = (struct Node*)malloc(sizeof(struct Node));
19
20     newNode->data = value;
21     newNode->left = NULL;
22     newNode->right = NULL;
23
24     return newNode;
25 }
26
27 // Preorder Traversal
28 void preorder(struct Node *root)
29 {
```

```
30     if(root != NULL)
31     {
32         printf("%d ", root->data);
33         preorder(root->left);
34         preorder(root->right);
35     }
36 }
37
38 // Inorder Traversal
39 void inorder(struct Node *root)
40 {
41     if(root != NULL)
42     {
43         inorder(root->left);
44         printf("%d ", root->data);
45         inorder(root->right);
46     }
47 }
48
49 // Postorder Traversal
50 void postorder(struct Node *root)
51 {
52     if(root != NULL)
53     {
54         postorder(root->left);
55         postorder(root->right);
56         printf("%d ", root->data);
57     }
58 }
```

```
55     postorder(root->right);
56     printf("%d ", root->data);
57 }
58 }
59
60 int main()
61 {
62     // Creating Binary Tree
63     struct Node *root = createNode(1);
64
65     root->left = createNode(2);
66     root->right = createNode(3);
67
68     root->left->left = createNode(4);
69     root->left->right = createNode(5);
70
71     // Traversals
72     printf("Preorder Traversal:\n");
73     preorder(root);
74
75     printf("\n\nInorder Traversal:\n");
76     inorder(root);
77
78     printf("\n\nPostorder Traversal:\n");
79     postorder(root);
80
81     return 0;
82 }
83
```

Preorder Traversal:

1 2 4 5 3

Inorder Traversal:

4 2 5 1 3

Postorder Traversal:

4 5 2 3 1

...Program finished with exit code 0

Press ENTER to exit console.